Experiment 7 : Emotion Detection from Text

Aim: Detect emotions (happy, sad, angry) in short text messages and compare two classifiers.

In [1]:
```
# A52 Dipanshu Ambilkar
import re
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
```

In [2]:
```
rng = np.random.default_rng(7)

EMOTION_WORDS = {
    "happy": ["happy", "joyful", "glad", "wonderful", "excited", "cheerfu
              "delighted", "thrilled", "amazing", "lovely", "great", "fan
              "blissful", "ecstatic", "merry", "jubilant"],
    "sad": ["sad", "gloomy", "depressed", "lonely", "miserable", "sorrowf
            "heartbroken", "tearful", "down", "hopeless", "blue", "melanc
            "dismal", "forlorn", "weepy", "dejected"],
    "angry": ["angry", "furious", "mad", "irritated", "outraged", "livid"
              "enraged", "infuriated", "annoyed", "hostile", "bitter",
              "resentful", "fuming", "seething", "wrathful", "cross"],
}
print("Emotions:", list(EMOTION_WORDS))
```

```
Emotions: ['happy', 'sad', 'angry']
```

In [3]:
```
TEMPLATES = [
    "I feel {w} today",
    "I am so {w} right now",
    "Feeling {w} and {w2} this morning",
    "This makes me feel {w}",
    "I woke up feeling {w}",
    "What a {w} day it has been",
    "I am {w} about everything",
    "Feeling completely {w} and {w2}",
]
print("Number of templates:", len(TEMPLATES))
```

```
Number of templates: 8
```

In [4]:
```
texts, labels = [], []
for emotion, words in EMOTION_WORDS.items():
    for _ in range(60):
        t = rng.choice(TEMPLATES)
        w1, w2 = rng.choice(words, 2, replace=False)
        texts.append(t.format(w=w1, w2=w2))
        labels.append(emotion)
print("Dataset size:", len(texts))
```

```
Dataset size: 180
```

In [5]:
```
print("Example text:", texts[0])
print("Its label:", labels[0])
```

```
Example text: Feeling completely lovely and great
Its label: happy
```

In [6]:
```python
def clean(text):
    text = text.lower()
    text = re.sub(r"http\S+|@\w+", "", text)
    text = re.sub(r"[^a-z ]", "", text)
    return text.strip()

print("Clean test:", clean("I am HAPPY!!! Check http://test.com @friend"))
```

```
Clean test: i am happy check
```

In [7]:
```python
texts = [clean(t) for t in texts]
print("Example after cleaning:", texts[0])
```

```
Example after cleaning: feeling completely lovely and great
```

In [8]:
```python
vectorizer = TfidfVectorizer(ngram_range=(1, 2))
X = vectorizer.fit_transform(texts)
print("TF-IDF feature matrix:", X.shape)
```

```
TF-IDF feature matrix: (180, 358)
```

In [9]:
```python
print("Number of word features:", len(vectorizer.get_feature_names_out()))
```

```
Number of word features: 358
```

In [10]:
```python
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3, random_state=7, stratify=labels)
print("Train size:", X_train.shape[0], "| Test size:", X_test.shape[0])
```

```
Train size: 126 | Test size: 54
```

In [11]:
```python
models = {"Logistic Regression": LogisticRegression(max_iter=1000),
          "Naive Bayes": MultinomialNB()}
print("Models to compare:", list(models))
```

```
Models to compare: ['Logistic Regression', 'Naive Bayes']
```

In [12]:
```python
scores = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    scores[name] = acc
    print(f"{name}: accuracy = {acc:.3f}")
```

```
Logistic Regression: accuracy = 0.870
Naive Bayes: accuracy = 0.796
```

In [13]:
```python
best = max(scores, key=scores.get)
print("Confusion matrix (" + best + "):")
print(confusion_matrix(y_test, models[best].predict(X_test),
                       labels=["happy", "sad", "angry"]))
```

```
Confusion matrix (Logistic Regression):
[[17  1  0]
 [ 2 16  0]
 [ 3  1 14]]
```

In [14]:
```python
for sentence in ["I am joyful and thrilled today",
                 "Feeling gloomy and tearful tonight",
                 "That insult made me absolutely furious"]:
```

```
    vec = vectorizer.transform([clean(sentence)])
    print(f'"{sentence}" -> {models[best].predict(vec)[0]}')
```

```
"I am joyful and thrilled today" -> happy
"Feeling gloomy and tearful tonight" -> sad
"That insult made me absolutely furious" -> angry
```